

1 Introduction

HSL_jll.jl is a pre-built version of libHSL designed to be easily used in the Julia ecosystem. Once installed, HSL_jll.jl allows calling the shared library, containing the C and Fortran routines of the HSL packages, directly from Julia. The HSL wrappers provided in the Julia interface HSL.jl become fully functional. HSL_jll.jl also offers a convenient way to employ HSL linear solvers MA27, MA57, HSL_MA77, HSL_MA86 and HSL_MA97 within the Ipopt.jl interface for the IPOPT nonlinear optimization solver.

HSL_jll.jl can be compiled with either libbstrampoline (LBT) or OpenBLAS. We provide HSL_jll.jl precompiled with LBT which requires at least Julia 1.9. LBT allows one to dynamically switch the BLAS and LAPACK backends between e.g. OpenBLAS, BLIS, Intel MKL or Apple Accelerate. HSL_jll.jl is precompiled for various operating systems (Windows, Mac, Linux, FreeBSD) and architectures (x64, arm64, ppc64).

For information on how to use each HSL routine available through this package please see the relevant documentation on [the HSL website](#). Where C interfaces are available, the C documentation should be used.

“One package to rule them all” – Alexis Montoisson

2 Use HSL_jll.jl in the Julia ecosystem

We strongly recommend using the Julia packages discussed below to interface with HSL_jll.jl. While calls to individual HSL routines are available via the Julia function `@ccall`, they are likely to be much more complicated than using equivalent functionality via HSL.jl.

2.1 Installation of the HSL_jll.jl package

To install the package HSL_jll.jl, you only need the following commands in the Julia REPL:

```
julia> ]
pkg> dev <hsl_jll_path>
```

Where `<hsl_jll_path>` is the HSL_jll.jl directory containing the Project.toml file.

Due to the security policy of the macOS operating system, Mac users may need to remove the quarantine before extracting.

```
xattr -d com.apple.quarantine HSL_jll.jl.v2024.11.28.zip
xattr -d com.apple.quarantine HSL_jll.jl.v2024.11.28.tar.gz
```

2.2 Load an LP64 BLAS and LAPACK backend

By default, Julia ships with only an ILP64 version of OpenBLAS as the BLAS and LAPACK backend. ILP64 libraries use 64-bit integers (necessary for indexing large arrays, with more than $2^{31} - 1$ elements), whereas the LP64 libraries index arrays with 32-bit integers in calls to BLAS and LAPACK.

The full catalogue of packages in LibHSL can only be linked with LP64 versions of BLAS and LAPACK as HSL only uses 32-bit integer index arrays. The user needs to manually load an LP64 BLAS and LAPACK backend except if HSL_jll.jl is used with [HSL.jl](#) or [Ipopt.jl](#).

A subset of HSL packages, used by [GALAHAD.jl](#), have been updated to allow the use of either LP64 or ILP64 BLAS and LAPACK calls. These can be called from outside of GALAHAD.jl through the `HSL_jll.libhsl_subset` and `HSL_jll.libhsl_subset_64` libraries directly with `@ccall`.

Because HSL packages heavily rely on BLAS and LAPACK routines, using an optimized backend for a particular platform can improve the performance.

If you have ReferenceBLAS32_jll.jl and LAPACKS32_jll.jl installed, switch to sequential and [reference version](#) of BLAS and LAPACK with:

```
# Load the sequential LP64 version of BLAS and LAPACK
using ReferenceBLAS32_jll, LAPACKS32_jll
LinearAlgebra.BLAS.lbt_forward(libblas32)
LinearAlgebra.BLAS.lbt_forward(liblapack32)
```

If you have `OpenBLAS32_jll.jl` installed, switch to OpenBLAS with:

```
# Load the LP64 version of OpenBLAS
using LinearAlgebra, OpenBLAS32_jll
LinearAlgebra.BLAS.lbt_forward(libopenblas)
```

If you have `blis32_jll.jl` and `LAPACKS32_jll.jl` installed, switch to BLIS with:

```
# Load the LP64 version of BLIS
using blis32_jll, LAPACKS32_jll
LinearAlgebra.BLAS.lbt_forward(blis32)
LinearAlgebra.BLAS.lbt_forward(liblapack32)
```

If you have `MKL.jl` installed, switch to MKL with:

```
# Load the LP64 version of Intel MKL
using MKL
```

If you are using macOS $\geq$ v13.4 and you have `AppleAccelerate.jl` installed, switch to Apple Accelerate with:

```
# Load the LP64 version of Apple Accelerate
using AppleAccelerate
```

Check what backends are loaded using:

```
using LinearAlgebra
LinearAlgebra.BLAS.lbt_get_config()
```

The user can also use their own LP64 version of BLAS and LAPACK. Note that LP64 and ILP64 versions can coexist if they have different symbols, such as a suffix `_64` in the naming conventions.

2.3 How to use `HSL_jll.jl` in Julia packages

A version of `HSL_jll.jl` based on a dummy `libHSL` has been precompiled with `Yggdrasil`. Therefore `HSL_jll.jl` is a registered Julia package and can be added as a dependency of any Julia package. The dummy version only has one C function `LIBHSL_isfunctional` that returns the Boolean `false`. The real version also exports this C function and returns the Boolean `true`.

```
using HSL_jll

function LIBHSL_isfunctional()
    @ccall libhsl.LIBHSL_isfunctional()::Bool
end

bool = LIBHSL_isfunctional()
```

This function makes it possible to determine if we have a dummy version or not, and thereby whether we can call the C and Fortran routines of the main version.

The package `HSL.jl` provides wrappers for all HSL packages with a C interface or those written in Fortran 77. Wrappers are Julia functions that call C and Fortran routines with the macro `@ccall`. Thus, the combination of `HSL.jl` and `HSL_jll.jl` allows one to avoid the low-level C and Fortran languages and use the majority of HSL packages as if they were packages developed in Julia. Note that `HSL.jl` also loads the LP64 version of OpenBLAS automatically for the user if needed.

For information on how to use each HSL routine available through this package please see the relevant documentation on [the HSL website](#). Where C interfaces are available, the C documentation should be used.

2.3.1 Usage with `HSL.jl`

A simple example where we use `HSL_MA97` to solve a random sparse symmetric system can be seen below.

All use is subject to licence.

<https://www.hsl.rl.ac.uk/>

```

using HSL
using LinearAlgebra
using SparseArrays

A = sprand(100, 100, 0.1)
A = A + A'
b = rand(100)

x = ma97_solve(A, b)

```

2.3.2 Usage with Ipopt.jl

For the Julia interface [Ipopt.jl](#), [HSL_jll.jl](#) must be used directly because it interacts with the [Ipopt_jll.jl](#) package under the hood.

```

using JuMP
import Ipopt, HSL_jll

# An optimization problem
model = Model(Ipopt.Optimizer)
@variable(model, x)
@objective(model, Min, (x - 2)^2)

# Load the HSL solvers
set_attribute(model, "hslilib", HSL_jll.libhsl_path)

# Use the linear solver MA57
set_attribute(model, "linear_solver", "ma57")
optimize!(model)

# Use the linear solver MA97
set_attribute(model, "linear_solver", "ma97")
optimize!(model)

```

The available HSL linear solvers are "ma27", "ma57", "ma77", "ma86" and "ma97". If no LP64 BLAS and LAPACK backend is loaded, [Ipopt.jl](#) loads [OpenBLAS32_jll.jl](#) automatically just like [HSL.jl](#).

We recommend using either sequential BLAS and LAPACK backends or a multithreaded version limited to one thread when employing the linear solvers "ma86", "ma87", or "ma97". These solvers already leverage parallelism via OpenMP, and enabling multiple threads in BLAS and LAPACK may result in thread oversubscription.

2.4 Cross-compilation

LibHSL can be easily cross-compiled with the Julia package [BinaryBuilder.jl](#). We provide the script `build_tarballs.jl` to regenerate [HSL_jll.jl](#) with different options or dependencies, such as METIS 4 instead of METIS 5. It can also be used to generate a static library `libhsl.a` for an application outside of Julia. Commands to cross-compile libHSL are available in the file `build_tarballs.jl`. Note that the Fortran compiler used to cross-compile libHSL is `gfortran`.

3 Bug reports and discussions

If you think you have found a bug, feel free to open an [issue](#). Useful suggestions and requests can also be opened as issues. If you would like to ask a question not suitable for a bug report, feel free to start a [discussion](#).

Acknowledgements

This work began while Alexis Montois was visiting Nick Gould, Jaroslav Fowkes and the [Computational Mathematics Group](#) at the [STFC Rutherford Appleton Laboratory](#) in December 2022 and July 2023. I want to express my appreciation to the HSL team for their invitations and their wonderful hospitality. I would also like to thank:

- Andrew Lister and Jaroslav Fowkes for their help with the documentation of libHSL;
- Stefan Vigerske for the [METIS](#) adapter;
- Eli Schwartz for their advices with the [Meson](#) build system;
- Dominique Orban, Geoffroy Leconte, Paul Raynaud, and Oscar Dowson for testing early versions of HSL_jll.jl;
- Mosè Giordano, and Elliot Saba for their work on [BinaryBuilder.jl](#), [libblastrampoline](#) and [Yggdrasil](#).